
Software Requirements Specification(SRS)

Ruth Akpalu, Didi Kone-Sow, Daksh Patel, Alisa Smanpongse,
Shawn Watkins, and Priscilla Cook
CS 350 Software Engineering
ODU Spring 2025

ODU Honor pledge

“I pledge to support the Honor System of Old Dominion University. I will refrain from any form of academic dishonesty or deception, such as cheating or plagiarism. I am aware that as a member of the academic community it is my responsibility to turn in all suspected violations of the Honor Code. I will report to a hearing if summoned.”

Table of Contents

1. Introduction.....	4
1.1 Purpose.....	4
1.2 Scope.....	4
1.3 Definitions, Acronyms, and Abbreviations.....	4
1.4 References.....	4
1.5 Overview.....	4
2. Overall Description.....	5
2.1 Product Perspective.....	5
2.2 Product Functions.....	5
2.3 User Characteristics.....	5
2.4 Constraints.....	5
2.5 Assumptions and Dependencies.....	5
3. Specific Requirements.....	6
3.1 External Interface Requirements.....	6
3.1.1 User Interfaces.....	6
3.1.2 Software Interfaces.....	6
3.1.3 Communication Interfaces.....	6
3.2 Functional Requirements.....	6
3.2.1 Lexical Analysis (FR1).....	6
3.2.2 Duplicate Code Detection (FR2).....	7
3.2.3 Unification Analysis (FR3).....	7
3.2.4 Reporting (FR4).....	7
3.2.5 Command-Line Invocation (FR5).....	7
3.3 Non-Functional Requirements.....	7
3.3.1 Performance Requirements.....	7
3.3.2 Error Handling & Edge Cases.....	7
3.3.3 Security Considerations.....	8
3.3.4 Usability Considerations.....	8
3.3.5 Extensibility.....	8
3.3.6 Test Plan and Verification.....	8
4. Appendices.....	9

1. Introduction

1.1 Purpose

The purpose of **DupDetector** is to analyze Java source code to identify near-duplicate code blocks that can be refactored into separate functions. This refactoring will help reduce code complexity, improve readability, and enhance maintainability.

1.2 Scope

DupDetector is a command-line Java application that:

- Reads Java source files (and directories recursively),
- Tokenizes the source code (ignoring comments, extra whitespace, and minor identifier differences),
- Detects repeated token sequences,
- Performs unification analysis to determine if sequences can be refactored by mapping identifiers and constants,
- Outputs a detailed report of refactoring suggestions ranked by potential token savings. It is implemented in Java 1.17 and runs on Windows, Linux, and macOS.

1.3 Definitions, Acronyms, and Abbreviations

- **CLI:** Command-Line Interface
- **JAR:** Java Archive
- **Token:** The smallest unit of a programming language's syntax (e.g., identifier, operator, reserved word).
- **Lexeme:** The actual character sequence that represents a token.
- **Unification:** The process of mapping lexemes from one token sequence to another to assess refactoring feasibility.

1.4 References

- IEEE 830-1998 Software Requirements Specification Standard
- Java Language Specification (latest version)
- Relevant academic literature on token-based code similarity and refactoring techniques

1.5 Overview

This document describes DupDetector's functionality, its external interfaces, functional and non-functional requirements, and additional requirements for error handling, security, usability, extensibility, and testing.

2. Overall Description

2.1 Product Perspective

DupDetector is a standalone tool designed to integrate into the developer's workflow. It does not modify code automatically but provides refactoring suggestions to improve source code quality. Its design leverages established lexical analysis techniques, similar to those found in compilers.

2.2 Product Functions

- **Lexical Analysis:** Tokenize Java source code while ignoring non-essential elements such as comments and excess whitespace.
- **Duplicate Detection:** Identify repeated token sequences across multiple files or non-overlapping sections of the same file.
- **Unification Analysis:** Evaluate whether detected sequences can be unified by consistent mapping of identifiers and constants.
- **Reporting:** Generate a detailed report that includes:
 - A list of all scanned files (with absolute paths and token counts),
 - A ranked list of refactoring suggestions detailing the token sequences, their locations, and the potential token savings.

2.3 User Characteristics

- **Developers:** Familiar with Java and interested in code quality improvements.
- **Software Engineers/Architects:** Focused on refactoring and maintaining large codebases.
- **Researchers:** Studying automated refactoring methods.
- **Project Managers:** Seeking to reduce technical debt in software projects.

2.4 Constraints

- Implementation must be in **Java 1.17**.
- DupDetector is distributed as a single **JAR file** and executed via the command line.
- Only Java source files (with a `.java` extension) in ASCII text are processed.
- The tool is non-interactive and solely CLI based.

2.5 Assumptions and Dependencies

- A compatible Java Runtime Environment (JRE) is available on the host machine.
- Source files are accessible and in the expected format.
- No interactive user input is required beyond the initial command-line parameters.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- **Command-Line Interface (CLI):**

Users interact with DupDetector via the terminal using the syntax:

- `java -jar DupDetector.jar nSuggestions [ -minLength k ] [ -maxParams n ] path1 [ path2 ... ]`
 - o `nSuggestions`: Maximum number of refactoring suggestions to display.
 - o `-minLength k`: Optional parameter to override the default minimum token sequence length.
 - o `-maxParams n`: Optional parameter to override the default maximum number of substitutions.
 - o `path1, path2, ...`: Paths to Java source files or directories.

3.1.2 Software Interfaces

- **Input:**
 - o Java source files and directories (recursively processed; only files ending in `.java` are considered).
- **Output:**
 - o A plain-text report listing:
 - All scanned files with their absolute paths and token counts,
 - Detailed refactoring suggestions with file paths, line numbers, column numbers, token sequences, and mapped lexemes.

3.1.3 Communication Interfaces

- No network or external communication is required by the system.
-

3.2 Functional Requirements

3.2.1 Lexical Analysis (FR1)

- Tokenize the content of Java source files while ignoring:
 - o Comments,
 - o Extra whitespace and indentation,
 - o Minor differences in identifiers and constants (subject to unification analysis).

3.2.2 Duplicate Code Detection (FR2)

- Identify repeated token sequences that meet or exceed a configurable minimum length.
- Ensure that sequences come from different files or non-overlapping portions of the same file.

3.2.3 Unification Analysis (FR3)

- Assess whether detected token sequences can be made equivalent via a consistent mapping of identifiers and constants.
- Only consider sequences “unifiable” if the mapping is consistent across the sequence.

3.2.4 Reporting (FR4)

- Generate an output report that includes:
 - o A list of scanned files (absolute paths and token counts),
 - o Refactoring suggestions that include:
 - The potential token savings (opportunity for improvement),
 - The token sequence length,
 - For each occurrence: file path, starting line number, starting column, and the unified lexemes.
- Order the report by:
 - o Descending order of opportunity (token savings),
 - o Alphabetical order of file paths,
 - o Ascending order of line numbers for sequences from the same file.

3.2.5 Command-Line Invocation (FR5)

- Support execution via the defined CLI syntax.
 - Accept optional parameters to override default values (e.g., minimum sequence length and maximum substitutions).
-

3.3 Non-Functional Requirements

3.3.1 Performance Requirements

- Process typical Java projects (e.g., 10,000+ lines of code) within 60 seconds.
- Use less than 500 MB of RAM during execution, even for large projects.

3.3.2 Error Handling & Edge Cases

- **Malformed Java Files:**
 - o Skip files that are malformed or unreadable.

- o Log appropriate error messages without terminating the overall process.
- **Directories with Non-Java Files:**
 - o Ignore non-Java files when scanning directories.
 - o Optionally log the presence of ignored files if verbose mode is enabled.
- **Infinite Loop Prevention:**
 - o Implement safeguards (e.g., maximum iteration counters, timeouts) during tokenization to prevent infinite loops.
 - o Generate warnings if abnormal processing durations are detected.

3.3.3 Security Considerations

- **Risk of Executing Arbitrary Code:**
 - o DupDetector only reads and tokenizes file content—it does not execute any Java code.
- **Memory Usage:**
 - o Implement memory management techniques (e.g., streaming and buffering) to handle large codebases.
 - o Enforce configurable memory usage limits, with graceful degradation or termination if limits are exceeded.

3.3.4 Usability Considerations

- **Verbose Mode:**
 - o Provide a verbose mode (triggered by a command-line flag) that outputs detailed processing logs and debugging information.
- **Color-Coded Output:**
 - o Optionally support color-coded terminal output to enhance readability (configurable based on terminal support).

3.3.5 Extensibility

- **Support for Additional Programming Languages:**
 - o Design the tokenization component with abstraction layers to enable future support for languages other than Java.
- **IDE Plugin Integration:**
 - o Architect the system to facilitate future development of plugins or extensions for popular IDEs.
 - o Define an API or standard output format to ease integration with IDE environments.

3.3.6 Test Plan and Verification

- **Unit Testing:**
 - o Conduct thorough unit tests for each module (tokenization, duplicate detection, unification analysis).

- o Include tests for edge cases such as empty files, files with only comments, and malformed code.
 - **System Testing:**
 - o Perform end-to-end tests on sample Java projects (including those with nested directories and mixed file types) to validate the entire workflow.
 - **Automated Test Coverage:**
 - o Integrate automated tests into the build process using frameworks such as JUnit.
 - o Maintain high test coverage to detect regressions early.
-

4. Appendices

Appendices may include sample input files, sample output reports, or additional reference material as needed.
